ENGINEERING RFC SPECIFICATION

Machine-to-Machine (M2M) API Key Engine

High-Throughput Cryptographic Key Provisioning & Scoped Ingress Gateway Architecture

Document ID	RFC-2026-M2M-AUTH-002	**Author**	Vicky Nishad (Java Backend Developer)
Status	**APPROVED & PRODUCTION-READY**	**Target Stack**	Java 21 LTS, Spring Boot 3.3+, Spring Security 6, MySQL 8.4

1. Executive Summary & Problem Formulation

Modern distributed applications rely on microservices, cron routines, event-driven webhooks, and third-party developer integrations. Traditional session-based authentication and interactive JSON Web Token (JWT) flows assume human interaction, rendering them poorly suited for automated machine authentication.

Standard industry implementations frequently store API keys in plaintext or execute full-table string evaluations, creating severe database exposure risks and catastrophic CPU bottlenecks during high request concurrency. This system establishes an enterprise-grade Ingress Gateway perimeter enforcing:

- **Zero Plaintext Storage:** Secret keys are generated with 256-bit cryptographically secure entropy and hashed using BCrypt (12 rounds). Secrets are exposed to callers strictly once upon provisioning.
- **Two-Part Key Partitioning:** By segregating keys into a public identifier (`key_id`) and private entropy (`secret`), database queries achieve constant time $\mathcal{O}(1)$ index lookups.
- **Fine-Grained Scope Enforcement:** Access is controlled via granular resource actions (`resource:action`) mapped into Spring Security authorities rather than broad, coarse-grained roles.
- **RFC 7807 Conformity:** All authentication and authorization failures return uniform Problem Details payloads.

2. High-Level Architecture (HLD) & Request Lifecycle

The security perimeter operates prior to MVC dispatcher invocation. Requests are intercepted, cryptographically verified, and decorated with security authorities inside an isolated request pipeline.

Phase	Component Responsible	Execution & Guarantees
1. Ingress Capture	`ApiKeyAuthFilter`	Extracts `X-API-KEY` header. Rejects malformed structures lacking delimiter `keyId.secret`.
2. Indexed Lookup	`ApiKeyRepository` (`JOIN FETCH`)	Performs $\mathcal{O}(1)$ B-tree lookup on `key_id` where `active = true`. Loads client entity eagerly to prevent session detachment.
3. Cryptographic Check	`PasswordEncoder` (BCrypt)	Evaluates incoming secret against stored hash in constant time. Checks temporal expiration (`expires_at > now()`).
4. Context Population	`SecurityContextHolder`	Binds client principal and transforms scope strings into `GrantedAuthority` objects (`SCOPE_orders:read`).
5. Policy Verification	`SecurityFilterChain`	Evaluates HTTP method and path matchers against granted scopes. Dispatches `403 Forbidden` if scopes are deficient.

3. Low-Level Design (LLD): Two-Part Token Partitioning

> **Partitioning Invariant:** `API_KEY = [ Public Routing Identifier (32 Chars) ] . [ Private Entropy (256-bit Base64) ]`

Storing monolithic API key hashes requires evaluating BCrypt over every active database row on incoming requests. By partitioning the token into a public routing key and private cryptographic secret, the engine achieves enterprise scalability:

- **Public Key ID (`key_id`):** Derived with format `ak_live_` + 16 random hexadecimal characters. Serves as a unique, clustered B-tree index in MySQL for instantaneous row location.
- **Cryptographic Secret (`secret`):** Generated using `java.security.SecureRandom` (32 bytes / 256 bits of entropy), encoded with URL-safe Base64. Persisted exclusively as a BCrypt digest. Plaintext is discarded immediately following key envelope dispatch.

4. Relational Database Schema & Data Integrity

The schema is managed strictly via versioned Flyway migrations (`V1__api_key_schema.sql`) with explicit referential integrity and index declarations.

Table	Column	Data Type	Constraints & Indexing
api_clients	`id`	BIGINT	Primary Key, Auto-Increment
	`client_name`	VARCHAR(100)	NOT NULL, Unique Index
	`contact_email`	VARCHAR(255)	NOT NULL
api_keys	`id`	BIGINT	Primary Key, Auto-Increment
	`client_id`	BIGINT	Foreign Key ➔ `api_clients(id)` (ON DELETE CASCADE)
	`key_id`	VARCHAR(32)	NOT NULL, Unique, B-Tree Index (`idx_key_id`)
	`secret_hash`	VARCHAR(255)	NOT NULL (BCrypt 12-Round Hash Digest)
	`scopes`	VARCHAR(500)	NOT NULL (Comma-delimited string)

5. Scoped Access Control Matrix (Principle of Least Privilege)

Traditional Role-Based Access Control (RBAC) grants overly broad administrative rights. Machine clients are restricted strictly through resource-action scopes:

Resource Path	HTTP Method	Required Scope	Spring Security Enforcement
`/api/v1/data/orders`	**GET**	`orders:read`	`hasAuthority("SCOPE_orders:read")`
`/api/v1/data/orders`	**POST**	`orders:write`	`hasAuthority("SCOPE_orders:write")`
`/api/v1/clients`	**POST**	*Public*	`permitAll()`
`/api/v1/keys/generate`	**POST**	*Public*	`permitAll()`

6. Security Threat Modeling & Mitigations

Threat Vector	Potential Impact	Architectural Countermeasure
Database Leakage	Direct exfiltration of MySQL database tables.	Only BCrypt hashes are stored. Plaintext secrets cannot be reversed from database dumps.
Timing Attacks	Statistical measurement of string matching latency.	BCrypt uses constant-time byte comparison routines to neutralize side-channel analysis.

Threat Vector	Potential Impact	Architectural Countermeasure
Privilege Escalation	Read-only client executes state-modifying actions.	Scoped authorities strictly isolate capabilities. Missing scopes yield `403 Forbidden`.
Stale / Zombie Credentials	Decommissioned services retain operational access.	Filter evaluates both `active == true` and explicit timestamp expiration.

7. Uniform Error Specification (RFC 7807)

The service standardizes API error payloads using Spring Boot 3 `ProblemDetail` semantics:

```
{ "type": "https://api.vickynishad.tech/errors/unauthorized", "title": "API Key Invalid or Missing", "status": 401, "detail": "A valid X-API-KEY header (keyId.secret) is required.", "instance": "/api/v1/data/orders" }
```

8. Architectural Decision Records (ADRs)

- **ADR-01 (Decoupled Configuration):** `PasswordEncoder` is isolated inside `ApplicationConfig` to eliminate circular startup bean cycles with `SecurityConfig` and `ApiKeyAuthFilter`.
- **ADR-02 (Stateless Eager Fetching):** Repository queries employ `JOIN FETCH k.client` to prevent Hibernate `LazyInitializationException` under high-performance `open-in-view: false` constraints.
- **ADR-03 (WebSecurityCustomizer Isolation):** OpenAPI documentation endpoints are whitelisted through `WebSecurityCustomizer.ignoring()`, bypassing filter overhead for testing interfaces.